

ICS. 14/10 T.

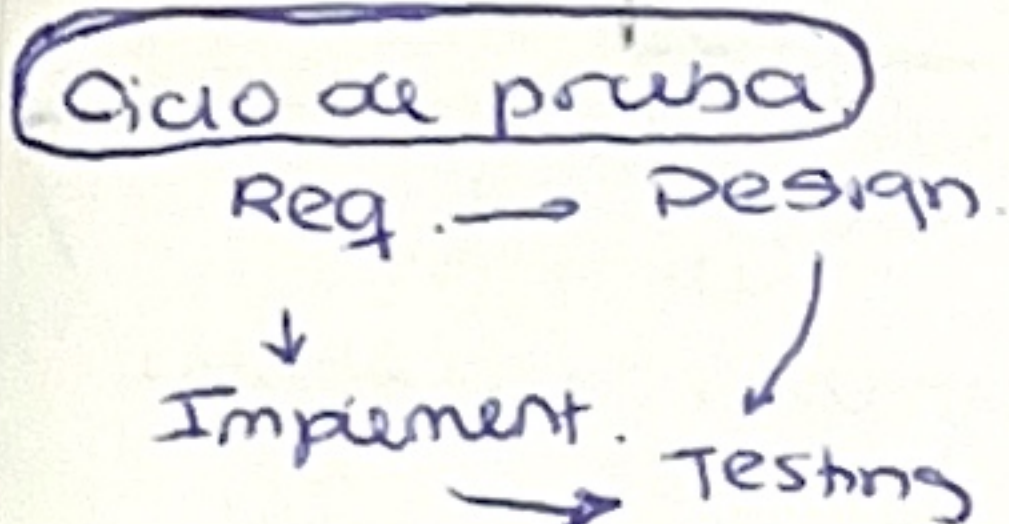
Testing en ambientes ágiles.

Antes hablamos de testing tradicional.

Ahora en ágil. $\approx$ TDD. $\Rightarrow$ Necesario aumentar testing.

Implementaciones híbridas $\Rightarrow$ Riesgos a se medían cosas se no van $\Rightarrow$ No se losía entregar stv de calidad en el tiempo esperado.

P/ adaptar la teoría a la organización.



Tendencia (muy fuerte en Ágil.)

Testing con la **automatización**

Manifiesto de testing.

Adaptación manifi. Ágil adaptado solo a testing.

- Valoramos $\oplus$ pruebas durante; $\oplus$ prevención de defectos; $\oplus$ entender el producto de que se trata; $\oplus$ Colaboro desde el principio; $\oplus$ Calidad del prod resp. de todo el equipo
- $\ominus$ y al final; $\ominus$ mal a hacer un check; $\ominus$ "ver como rompo todo"; $\ominus$ No es culpa de testing
- $\Rightarrow$ así no es lo tradicional xq no se puede hacer testing.

Apuntamos a.

Integrar testing

Ador $\oplus$ chiguitos de hace p/ cada user integrando testing desde el principio. adelantar trabajo testing. y estar incertis en las vs (confirmación)

CA. PA. $\Rightarrow$ 1 criterio muchas pruebas de aceptación.

Principios.

- todo el mundo hace testing
- feedback. menor latencia/demora
- código limpio: calidad.
- reducir sobrecarga de doc. de pruebas.

detalles necesarios.

$\Rightarrow$ Las pruebas son EJEMPLOS $\rightarrow$ detallado y específico. $\Rightarrow$ entados en la user/res. de mueren al testing. de automatizan $\ominus$ carga operativa.

$\downarrow$

50% parte del bod.

Practicar concretal.

$\Rightarrow$ Materializar lo que dice el manifiesto.

- probar uid. adom.
- TDD. o ATDD.
- usar pruebas exploratorias.
- excelente gestión de configuración $\rightarrow$ control de versiones.

Básico mínimo que tiene que existir

prácticas continuas

CI $\hookrightarrow$ integración
CD $\hookrightarrow$ delivery
ED $\hookrightarrow$ deployment

continuas. $\approx$ Automatización de los niveles de prueba.

Más allá de adaptar prácticas.

$\neq$ ser ágil.

$\hookrightarrow$ Actitud con la que encaramos las actividades que son las mismas act p/ hacer stv

- en 3 momento
- por partes, todo junto

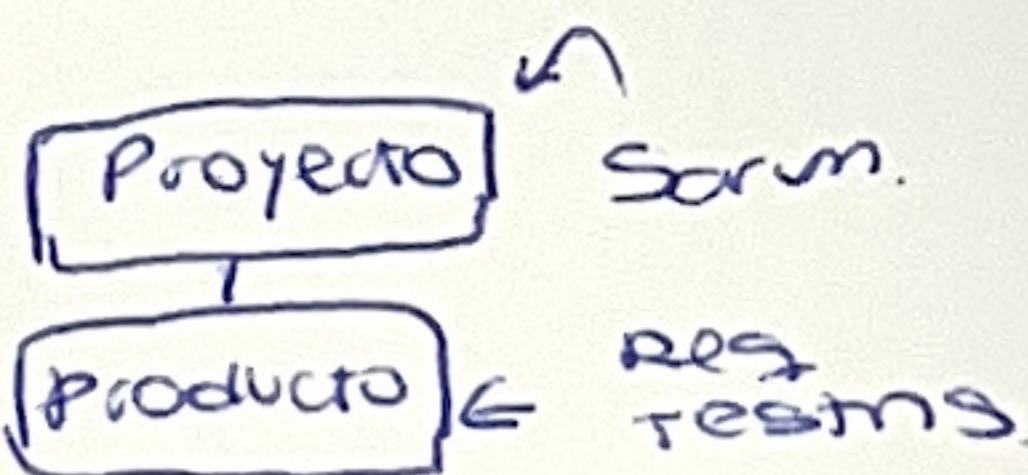
Interacción entre actividades $\Rightarrow$ "todas juntas" granularidad $\rightarrow$ 1 carril. en ágil.

pero arrancar cuando un producto no esta 100% especificado pero ∇ TIENE Q ESTAR CLARO

Actitud de todo el equipo

$\oplus$ calidad del prod resp. de todo el equipo

$\ominus$ No es culpa de testing



1. TDD automatización del testing unitario.

2. Nivel de testing $\rightarrow$ Integración en CI

3. en delivery $\rightarrow$ Nivel de sistema o versión

4. en development $\rightarrow$ nivel de aceptación

ATDD: A nivel de users
de negocio.
no manual
y lo hacen
automatizado.

Prueba de regresión → volver a ejecutar todo, como si fuera
algo nuevo.

sin regresión → ejecutar la lista de defectos.
lo que no se aprobó no se
vuelve a probar. errores
nuevos.

La configuración
el manual } tiene un corto
configuración en s converge
(3 ciclos de prueba)
s, t. → según complejidad.
herramiental
formación en los tests.
automatización

¿por qué no lo hacemos todo?

Vínculo.
Tester.

Gradientes del testing así → como llevar adelante
las pruebas

original ⊕ orientada a }
tecnología: req. N.Func.
negocio: cumplir las users. } filas.

una no ayudan a CONSTRUIR. (izq.) }
el producto. } columnas.

pruebas que ayudan a VALIDAR. (der)
si el producto es correcto

Automatización

⊕ rápido.

⊕ cobertura: cantidad de pruebas
que cubren sobre la totalidad

⊕ precisión

regresión se planea hacer.

reducción de costos a mediano/largo plazo (alto costo
de prueba.)
↳ tiene costos iniciales altos.

Pirámide.

Base fuerte.
de pruebas
unitarias.

pruebas funcionales.

pruebas tipos.
procesos de req.

⊕ pruebas manuales.

Niveles
pruebas.

• Unitario = 1 componente

• Integración - varios
componentes.

• Adaptación - user
(último nivel) por el
entorno

• Sistemas - puede no
ser end to
end.

se
prueba lo
mismo.
pero la
de acept.
la hace el
usuario.